

BabyTalk Architecture Deep Dive

BabyTalk App — Architecture Deep Dive

Document version: July 2026 (expanded walkthrough)

Branch: `feature/google-drive` (waveform rescope)

Audience: Developer reference — start with **§4** for the file-tied pipeline tour

1. Executive summary

BabyTalk is a React Native app for recording baby sounds, tagging moments, visualizing waveforms, and exporting sessions locally. The architecture deliberately **separates concerns**:

Layer	Responsibility	Where it lives
Audio engine	Record/play <code>.m4a</code> , position + metering	<code>react-native-audio-recorder-player</code> (Swift/Kotlin)
Bridge	Native ↔ JS events (~50ms)	<code>index.ts</code> listeners + <code>rn-recordback</code> / <code>rn-playback</code>
Waveform capture	Events → sparse <code>{tMs, avgDb, peakDb}</code>	<code>RecordScreen.tsx</code> + <code>waveform/storage.upsertSample</code>
Waveform store	Persist payload + tags	<code>serializeWaveform</code> → <code>db.ts</code> → SQLite
Waveform renderer	Dense bars, cursor, tags, fixed Y	<code>Waveform.tsx</code> + <code>waveform/scale.ts</code>
Playback	Load payload, 8s window, play sync	<code>PlaybackScreen.tsx</code>

The audio engine is **not** a waveform library. Visualization lives in `src/waveform/` and `src/components/Waveform.tsx` . Capture wiring is in the screens, not a separate `waveform/capture` module.

Read next: §3 diagram → **§4 hop-by-hop walkthrough** → §5–§6 design notes.

Near-term roadmap: validate waveform on device → structured export → Google Drive backup → offline/cloud processing → optional on-device ML.

2. Technology stack

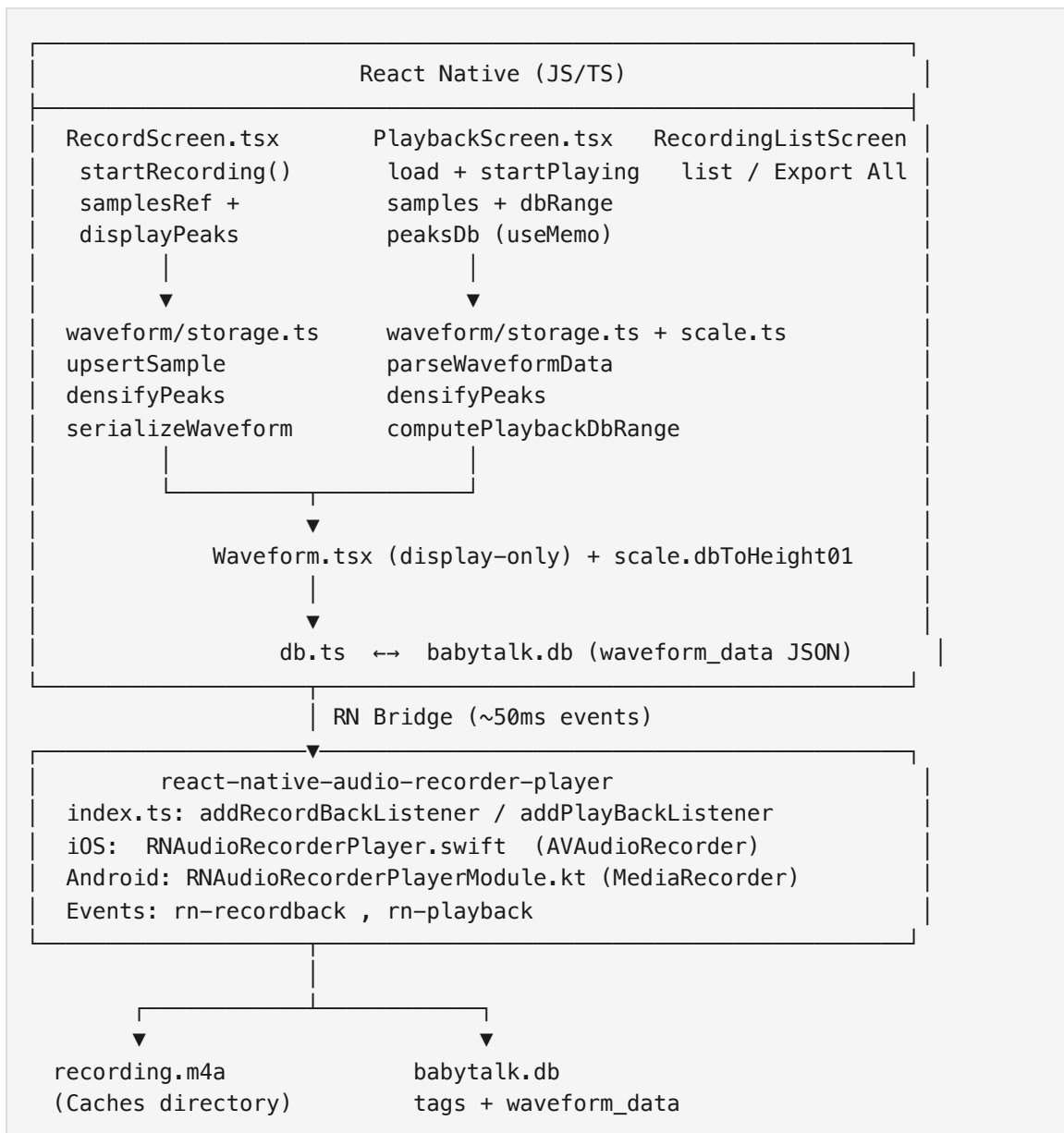
Component	Choice	Role
React Native 0.78.1	Cross-platform shell	UI, navigation, business logic
TypeScript 5.0.4	Language	Type safety
Hermes	JS engine	Runtime
react-native-audio-recorder-player	Local fork	Native record/play + metering callbacks
react-native-sqlite-storage	SQLite	Metadata, tags, waveform JSON
react-native-fs	File I/O	Resolve audio paths, export files
react-native-share	Share sheet	Export recording + tags
@react-navigation/native-stack	Navigation	Record → List → Playback

Why React Native (not pure Swift)?

- Single codebase for iOS and Android
 - Fast iteration with Metro hot reload
 - Heavy lifting (microphone, filesystem) delegated to native modules
 - Trade-off: bridge latency (~50ms ticks) vs 60fps native UI (Voice Memos)
-

3. System diagram

There is **no** separate `waveform/capture` module. Capture wiring lives in `RecordScreen.tsx`; helpers (`upsertSample` , `densifyPeaks` , `parse/serialize`) live in `src/waveform/storage.ts` .



Pipeline we follow below:

engine → bridge → capture → store → render → playback

4. End-to-end walkthrough (tied to files)

This section follows **one tap of Record** through the stack, then **Save**, then **Playback**. File paths are relative to the app root unless noted.

Hop overview

Hop	Layer	Primary files
1	Engine	<code>react-native-audio-recorder-player/ios/RNAudioRecorderPlayer.swift</code>
2	Bridge	same Swift + <code>react-native-audio-recorder-player/index.ts</code>
3	Capture	<code>src/screens/RecordScreen.tsx</code> + <code>src/waveform/storage.ts</code> (<code>upsertSample</code>)
4	Store	<code>RecordScreen.tsx</code> (<code>handleSave</code>) → <code>storage.serializeWaveform</code> → <code>src/db.ts</code>
5	Render (live)	<code>storage.densifyPeaks</code> → <code>src/components/Waveform.tsx</code> + <code>src/waveform/scale.ts</code>
6	Playback	<code>PlaybackScreen.tsx</code> → <code>parse/scale/densify</code> → <code>Waveform.tsx</code> + play-back listener

Hop 1 — Engine (native record + meter)

What starts it (JS): `RecordScreen.startRecording()` calls:

1. `audioRecorderPlayer.setSubscriptionDuration(SUBSCRIPTION_SEC)` — `0.05` from `src/waveform/config.ts`
2. `audioRecorderPlayer.startRecorder('recording.m4a', undefined, true)` — third arg enables metering

What native does (iOS): In `RNAudioRecorderPlayer.swift` :

- `startRecorder(...)` creates `AVAudioRecorder`, sets `isMeteringEnabled`, starts recording to Caches, then `startRecorderTimer()`.
- `startRecorderTimer()` schedules a repeating `Timer` with interval `subscriptionDuration` (50ms after the JS call above).
- Each tick calls `updateRecorderProgress(timer:)` :

```
// RNAudioRecorderPlayer.swift - updateRecorderProgress
audioRecorder.updateMeters()
currentMetering = audioRecorder.averagePower(forChannel: 0)
currentPeakMetering = audioRecorder.peakPower(forChannel: 0)
// currentPosition = audioRecorder.currentTime * 1000
sendEvent(withName: "rn-recordback", body: status)
```

Android analog: `RNAudioRecorderPlayerModule.kt` posts a `Runnable` every `subsDurationMillis`, reads `MediaRecorder.maxAmplitude`, converts to dB, and puts the **same** value in both `currentMetering` and `currentPeakMetering` (no separate peak API).

What the engine owns: writing `.m4a`, session clock (`currentPosition`), loudness snapshots (dB).

What it does not own: bar layout, SQLite, tags, Y-scale, React UI.

Module path caveat: `package.json` points at `file:../react-native-audio-recorder-player`. CocoaPods resolves via `node_modules` symlink. Swift/Kotlin changes need a **full native rebuild**, not Metro Fast Refresh alone.

Hop 2 — Bridge (native event → JS listener)

Bridge: React Native serializes a small map from native → JS. Native cannot call React functions directly; it emits a named event.

Emit (native): `sendEvent(withName: "rn-recordback", body: { isRecording, currentPosition, currentMetering, currentPeakMetering })`.

Subscribe (JS): `react-native-audio-recorder-player/index.ts` — `addRecordBackListener(callback)` :

- iOS: `NativeEventEmitter(RNAudioRecorderPlayer).addListener('rn-recordback', callback)`
- Android: `DeviceEventEmitter.addListener('rn-recordback', callback)`

Who registers the callback: `RecordScreen.tsx` immediately after `startRecorder` succeeds:

```
audioRecorderPlayer.addRecordBackListener((e) => { /* Hop 3 */ });
```

Payload fields:

Field	Role
<code>currentPosition</code>	ms on the recorder clock → timer, tags, bin index
<code>currentMetering</code>	averagePower (iOS) — stored as <code>avgDb</code>
<code>currentPeakMetering</code>	peakPower (iOS) — stored as <code>peakDb</code> , used for bar height

Alignment rule: one sample per callback, snapped to **50ms** bins (`BAR_MS`). Callbacks can skip under load; the **file** stays continuous, the **metering stream** can have holes — that is why capture is sparse.

Playback side of the bridge: `addPlayBackListener` stores a JS callback; native emits `rn-playback` with `{ currentPosition, duration, isFinished }` while playing. Same bridge idea, different event name.

Hop 3 — Capture (JS turns events into sparse samples)

File: `src/screens/RecordScreen.tsx` (listener body) + `src/waveform/storage.ts` (`upsertSample`).

On every `rn-recordback` :

1. `tMs = e.currentPosition`; `setRecordMs(tMs)` (drives timer + re-render cadence).
2. Read `avgDb` / `peakDb` from metering fields (fallback `CAPTURE_DB_MIN` = -160).
3. `upsertSample(samplesRef.current, { tMs, avgDb, peakDb })`.

`upsertSample ( storage.ts ):`

- Snaps `tMs` to nearest `BAR_MS` bin: `round(tMs / 50) * 50`.
- If that bin already exists, keeps the **louder** avg/peak (`Math.max`).
- Otherwise appends (or inserts if rare out-of-order).

In-memory shape during recording:

```
samplesRef: WaveformSample[] // sparse, mutated in place – source of truth
                             for Save
displayPeaks: number[]       // React state – dense peak dB for the last 3s
                             only
```

Types live in `src/waveform/types.ts` :

```
type WaveformSample = { tMs: number; avgDb: number; peakDb: number };
```

Capture stores **raw dB** across the full -160...0 range. Display mapping happens later (Hop 5 / 6).

Hop 4 — Store (persist payload + tags)

When: user taps Save → `RecordScreen.handleSave` → stop recorder if needed → prompt for session name.

Build payload:

```
const payload = emptyWaveformPayload(); // version + barDurationMs: 50
payload.samples = samplesRef.current.slice();
waveformData: serializeWaveform(payload) // JSON.stringify
```

WaveformPayload v2 (written into SQLite `recordings.waveform_data`):

```
{
  "version": 2,
  "barDurationMs": 50,
  "samples": [
    { "tMs": 0, "avgDb": -45.2, "peakDb": -38.1 },
    { "tMs": 50, "avgDb": -42.0, "peakDb": -35.0 }
  ]
}
```

Why metadata? Playback must know bins are 50ms. Older code guessed `durationMs / array.length`, which breaks on sparse samples or bin-size changes. `version` + `barDurationMs` are the contract.

DB write: `src/db.ts` → `addRecording({ filename, sessionName, durationMs, waveformData })` inserts into `recordings`. Then `addTag(...)` for each live tag.

Also on disk: the `.m4a` from Hop 1 (Caches). DB stores the **filename**; list/playback resolve the path via `react-native-fs` when needed.

Legacy: `parseWaveformData` still accepts old flat `number[]` (normalized 0–1) and approximates dB for display — new recordings always write v2.

Hop 5 — Render while recording (live tail)

Still inside the same record-back listener, after `upsertSample`:

```
setDisplayPeaks(
  densifyPeaks(samplesRef.current, tMs - RECORD_WINDOW_MS, tMs, BAR_MS)
);
```

densifyPeaks (storage.ts): sparse samples → dense peakDb[] for [startMs, endMs) . Missing bins → CAPTURE_DB_MIN (silence). startMs may be **negative** early in a session so the window is always 3s and “now” sits at the **right** edge (left-padded silence).

Constants: RECORD_WINDOW_MS = 3000 → **60 bars** at 50ms.

UI: RecordScreen passes into Waveform.tsx :

Prop	Value
peaksDb	displayPeaks (already densified for the window)
windowMs	RECORD_WINDOW_MS (3s)
progressMs	recordMs
mode	'rolling'
cursorMode	'pinned' (cursor at right edge)
dbRange	RECORD_DB_RANGE from scale.ts → fixed -60...0

Waveform.tsx : display-only. Maps each peak dB with dbToHeight01(db, dbRange) (scale.ts): noise floor (~-58 dB) → flat bar; no live autoscale. Draws tag markers from tagTimestampsInWindow .

Render cadence ≈ callback cadence (~50ms), because setRecordMs / setDisplayPeaks run on each event.

Hop 6 — Playback (load store → fixed Y → rolling window → play)

Load (mount): PlaybackScreen.loadRecordingWaveform :

1. SQL: SELECT duration_ms, waveform_data FROM recordings WHERE id = ?
2. parseWaveformData(row.waveform_data) → { samples, barDurationMs }
3. setDbRange(computePlaybackDbRange(payload.samples)) — **once per session**

computePlaybackDbRange (scale.ts): sort peakDb , take **95th percentile**, map that level to **80%** of bar height (TARGET_PEAK_FRACTION). Shrieks above that clip at the top; quieter speech stays visible. No breathing autoscale during play.

Play: startPlayer(filePath) then addPlayBackListener → setPlaybackMs(e.currentPosition) (and duration). Same engine, rn-playback events.

Window for drawing: `peaksDb` `useMemo` in `PlaybackScreen` :

```
densifyPeaks(samples, playbackMs - PLAYBACK_WINDOW_MS, playbackMs,
             barDurationMs)
// PLAYBACK_WINDOW_MS = 8000 → 160 bars @ 50ms
```

UI: `Waveform` with `cursorMode="follow"`, `dbRange` from step 3, tags from `getTagsForRecording`. Optional toggle to `'full'` densifies `0...duration` (may downsample only if bars exceed pixel width).

Sync note: `playbackMs` is the player clock delivered over the bridge (~50ms ticks + React render). Same module as recording → no dual-clock drift; not sample-accurate.

Walkthrough summary (one sample's life)

```
AVAudioRecorder (file + meters)
  → Timer 50ms → updateRecorderProgress
  → sendEvent("rn-recordback")
  → index.ts addRecordBackListener
  → RecordScreen listener
  → upsertSample(samplesRef)           // sparse raw dB
  → densifyPeaks → setDisplayPeaks     // live 3s tail
  → Waveform.tsx + RECORD_DB_RANGE    // fixed -60..0
  → [Save] serializeWaveform → db.addRecording
  → [Open] parseWaveformData → computePlaybackDbRange
  → densifyPeaks(8s) → Waveform + follow cursor
  → startPlayer + rn-playback → playbackMs
```

5. Design principles and constants

Locked principles (July 2026)

1. **Separate engine from visualization** — library records/plays; `src/waveform/*` + `Waveform.tsx` own viz
2. **Capture and display both at 50ms** (may diverge later)
3. **Render cadence aligned to callbacks** (~50ms state updates)
4. **Fixed Y scale per session** — record: -60...0; playback: percentile once at load
5. **Path B later** — regenerate peaks from `.m4a` (file analysis), not only live metering

Constants (`src/waveform/config.ts`)

Constant	Value	Meaning
<code>SUBSCRIPTION_SEC</code>	0.05	Native callback interval
<code>BAR_MS</code>	50	One bar / one bin = 50ms
<code>RECORD_WINDOW_MS</code>	3000	Live tail: 60 bars
<code>PLAYBACK_WINDOW_MS</code>	8000	Rolling playback: 160 bars
<code>CAPTURE_DB_MIN/MAX</code>	-160 / 0	Full range stored
<code>RECORD_VISIBLE_DB_MIN/MAX</code>	-60 / 0	Fixed display while recording
<code>PLAYBACK_SCALE_PERCENTILE</code>	0.95	Session Y from 95th percentile peak
<code>TARGET_PEAK_FRACTION</code>	0.8	That percentile → 80% bar height
<code>DISPLAY_NOISE_FLOOR_DB</code>	-58	At/below → silence bar

6. Sparse storage and densify (why this shape)

Why sparse `{ tMs, avgDb, peakDb }[]` ?

A dense array assumes slot `i` = time `i × 50ms` . Dropped callbacks leave holes or force inventing silence in the stored array, and `length × 50 ≈ duration` breaks. Sparse storage only records measured ticks; display densifies on the fly.

Why densify only for render?

`densifyPeaks` fills missing bins with silence **for drawing**. The DB keeps the honest sparse list. You can change window size or display metric (peak vs avg overlay later) without rewriting stored data.

Pixel budget (playback, no downsampling)

`bars = windowSeconds × 20` (at 50ms). On ~390px width with `minBarPx = 1` , **8s → 160 bars** fits. Longer full-clip views may downsample in `Waveform.tsx` only when `peaksDb.length > width / minBarPx` .

7. Parameter map (visualization success)

Layer 1 — Signal acquisition (native)

Parameter	Source	Notes
Callback interval	<code>setSubscriptionDuration(0.05)</code>	Max ~20 updates/sec
averagePower	iOS AVAudioRecorder	Stored, not displayed yet
peakPower	iOS AVAudioRecorder	Displayed
Capture dB range	-160..0	Full range stored

Layer 2 — Capture & storage

Parameter	Value	Notes
Bin size	50ms	Matches callbacks
Structure	Sparse { <code>tMs</code> , <code>avgDb</code> , <code>peakDb</code> }	Robust to dropped ticks
Persisted format	WaveformPayload JSON	version + <code>barDurationMs</code>

Layer 3 — Time axis (X)

Parameter	Recording	Playback
Window	3s tail	8s rolling
Bars in window	60	160
Cursor	Pinned right	Follows playhead
Downsampling	None (fits screen)	None for 8s @ 50ms

Layer 4 — Amplitude axis (Y)

Phase	Strategy
Recording	Fixed -60..0 dB visible range
Playback	95th percentile peak → 80% height; fixed for session
Noise floor	-58 dB → render as silence

Layer 5 — Sync

Check	Status
Same audio clock for position	Yes (native module)
Bridge latency	Up to ~50ms + React render
Tag timestamps vs waveform	Aligned when barDurationMs metadata correct

8. Path A vs Path B (waveform fidelity)

Path A — Live metering (current)

- One loudness number every 50ms while recording
- Summarizes ~2,205 PCM samples at 44.1kHz (iOS)
- **VU-meter trace**, not sample-accurate waveform
- Good enough for live feedback and coarse playback shape

Path B — File analysis (Voice Memos style, future)

- After stop: decode `.m4a` → PCM → compute peak/RMS per 50ms window
- Reflects **actual audio content**
- Can regenerate for old recordings
- Libraries: `react-native-audio-analyzer`, native `AVAssetReader`, or Simform waveform (Option D)

Audio file temporal resolution

M4A/AAC typically encodes ~44,100 samples/second. You **hear** full resolution; live metering **shows** 20 envelopes/second.

9. Database and files

SQLite schema

recordings

Column	Type	Purpose
id	INTEGER PK	
filename	TEXT	e.g. recording.m4a
session_name	TEXT	User label
created_at	INTEGER	Unix ms
duration_ms	INTEGER	Session length
waveform_data	TEXT	WaveformPayload JSON

tags

Column	Type	Purpose
id	INTEGER PK	
recording_id	FK	
timestamp_ms	INTEGER	Moment in recording
label	TEXT	e.g. hungry, tired

File storage

- iOS: app Caches directory
- Format: M4A (AAC)
- Export: Share sheet (single or Export All with JSON manifest)

10. Screen map

Screen	File	Key behaviors
Record	RecordScreen.tsx	Record, tag live, 3s waveform tail, save
List	RecordingListScreen.tsx	Search, sort, Export All
Playback	PlaybackScreen.tsx	Play, 8s waveform, tag, export

Shared components

- `Waveform.tsx` — bars, cursor, tag markers
- `CircularRecordButton` / `CircularPlayButton`
- `RecordingLayout` — common chrome

11. Comparison to Voice Memos / phone call bar

Feature	Voice Memos / system UI	BabyTalk (current)
Render thread	Native UI / 60fps	React state ~20Hz
Live input	High-frequency native metering	50ms bridge callbacks
Playback waveform	From decoded file	From stored metering (Path A)
Y scale	Fixed / slow AGC	Fixed per session (implemented)

12. Roadmap (from planning sessions)

Done (waveform rescope)

- ✓ 50ms sparse capture with raw dB
- ✓ peak + avg metering from native
- ✓ 3s record tail, 8s playback window
- ✓ Fixed Y scale
- ✓ WaveformPayload v2 metadata

Next

- ☐ Device test checklist (simulator / iOS 18 device / Tahoe + Xcode 26 for iOS 26.5)
 - ☐ Structured export schema (waveform + tags + manifest for cloud jobs)
 - ☐ Google Drive upload + sync status in DB
 - ☐ Path B: file-based peak envelope
 - ☐ Cloud or local ML pipeline
 - ☐ Optional: averagePower overlay on peak bars
-

13. Build and deploy notes

iOS device requirements (July 2026)

- **iOS 26.5 device** requires **macOS Tahoe 26.2+** and **Xcode 26.5+** (Developer Disk Image mismatch on Xcode 16.2)

- **Simulator** (iPhone 16, iOS 18.3) works for UI flow; mic is limited
- Native module changes require **Clean Build** in Xcode, not Metro reload alone

Known warnings

- `react-native-sqlite-storage` invalid `dependency.platforms.ios.project` — CLI metadata warning; usually harmless if DB works

14. File reference

```
src/
├─ waveform/
│   ├── config.ts      # Constants
│   ├── types.ts       # WaveformSample, WaveformPayload
│   ├── storage.ts     # upsert, densify, parse, serialize
│   └─ scale.ts        # dbToHeight01, computePlaybackDbRange
├─ components/
│   └─ Waveform.tsx    # Display-only renderer
├─ screens/
│   ├── RecordScreen.tsx
│   ├── PlaybackScreen.tsx
│   └─ RecordingListScreen.tsx
└─ db.ts               # SQLite API

react-native-audio-recorder-player/ # In-repo copy (import path)
../react-native-audio-recorder-player/ # npm/CocoaPods linked copy
```

15. Glossary

Term	Definition
Callback	Native → JS event on a timer (record-back / playback)
Bridge	RN message channel between JS and native
DDI	Developer Disk Image — Xcode uses to debug on physical device
Densify	Convert sparse samples to fixed 50ms bar array for drawing
Metering	Loudness in dB from recorder, not PCM samples
Sparse storage	List of {tMs, dB} only where samples exist
VU trace	Path A envelope from live metering

Generated for BabyTalk App architecture review. Update when Path B, Drive sync, or ML pipeline lands.